Strata — Hướng dẫn BA

Dành cho Business Analyst

strata-kb v1.35.0 · 2026-11-08

1. Tài liệu này dành cho ai

Bạn viết yêu cầu: ticket, user story, tiêu chí chấp nhận và, với tính năng lớn, các mission plan chia nhỏ chúng. Một ticket bạn hoàn thành là **Dev-ready**: mọi khẳng định chạm tới một tiêu chuẩn đều đến từ một tài liệu thật, ở một phiên bản xác định, và vẫn kiểm chứng được nhiều tháng sau khi có người hỏi nó từ đâu ra.

Vấn đề của bạn không phải là viết. Đó là **bám nguồn (grounding)**: Strata cho bạn khả năng tìm kiếm trên tri thức đã publish, **trích dẫn ghim phiên bản** và **cổng Definition-of-Ready** (`kb ticket lint`, `kb mission lint`), để một ticket không thể đến tay developer với trích dẫn hỏng hoặc thiếu. Bạn **publish kế hoạch** lên hub khi nó sẵn sàng, và **amend** mission hay ticket tại chỗ khi tri thức, kiến trúc, phạm vi hoặc tiến độ giao hàng thay đổi.

Repo BA có làm	Repo BA không làm
Đọc tri thức đã publish trên hub	Ingest tài liệu hay tóm tắt bất cứ thứ gì
Trích dẫn nó, ghim vào một commit của hub	Publish nội dung tri thức
Quản lý phiên bản ticket và mission trong Git, và chặn chúng ở CI	Đẩy sang công cụ quản lý issue của bạn: trợ lý không bao giờ tự mở ticket cho bạn
Publish kế hoạch của chính nó, `<repo_id>-plan`, khi bạn publish hoặc merge	Publish bản nháp, hay bất cứ thứ gì bạn chưa đánh dấu sẵn sàng
Chạy chế độ full-auto khi bạn bật công tắc (mục 4)	Tự merge hay tự ship bất cứ thứ gì nếu không bật

Repository của bạn **chỉ đọc** đối với cơ sở *tri thức*. Bạn trích dẫn, không bao giờ đóng góp. Đó là chủ ý: nó giữ cho yêu cầu truy vết được tới một phiên bản tri thức mà không cho quy trình yêu cầu cách nào để thay đổi chính tri thức. Thứ duy nhất bạn gửi lên hub là kế hoạch của mình: các mission và ticket bạn đã review, dưới dạng một tài liệu sinh tự động. Nó nằm cạnh tri thức và không bao giờ sửa tri thức.

Các khái niệm dùng chung, như các lớp L0 đến L3 và trích dẫn `kb-context`, được giải thích trong tài liệu kiến trúc.

Slash command. Mọi slash command trong tài liệu này đều có cho Claude Code, GitHub Copilot và Cursor, trừ các ngoại lệ sau:

- `/kb-publish` trong Claude Code là một skill (`.claude/skills/kb-publish/`), không phải file trong `.claude/commands/`. Bạn vẫn gõ `/kb-publish`.
- Repo BA không có `/kb-ingest`, `/kb-summarize`, `/kb-init` hay `/kb-l3-fix`: nó không bao giờ ingest hay tóm tắt.

Phía Dev của quá trình bàn giao (lấy ticket của bạn, hỏi bạn câu hỏi, chế độ auto của chính Dev) nằm trong Hướng dẫn cho Developer.

2. Thiết lập lần đầu

2.1 Cài đặt và dựng khung

```
pip install strata-kb
kb init --kind ba
```

`kb init --kind ba` ghi config, cấu hình MCP, các slash command và skill cho cả ba trợ lý, `.claude/settings.json` (được gộp vào file của bạn), `tickets/` và `missions/`, các template trong `docs/`, các file ghi đè chất lượng `*.local.md` của bạn (tạo một lần, không bao giờ làm mới) và bốn workflow: `kb-ticket-lint.yml`, `kb-doctor.yml`, `kb-publish.yml` và `kb-publish-request.yml`. Chạy lại lệnh này là an toàn (6.2).

2.2 Trỏ tới hub và đăng ký repo

```
# .kb/config.yaml
kind: ba
hub: https://github.com/acme/kb-hub.git
repo_id: acme-requirements
intake: https://kb-intake.acme.com
```

- **`hub`** là nơi `kb ticket lint`, `kb query` và `kb resolve` đọc tham chiếu.
- **`repo_id`** đặt tên cho tài liệu bạn publish, `<repo_id>-plan`, và phải bằng id của repo này trong `federation/registry.yaml` của hub. Hãy nhờ người vận hành hub thêm `owner/repo` của bạn dưới `repos:` ánh xạ tới id đó, và để trống `workflow:`: intake so sánh nó với `job_workflow_ref` đầy đủ, vốn chứa tag publish, nên một pin không bao giờ khớp. Việc publish bị từ chối cho tới khi bạn được đăng ký. Branch protection trên hub cộng với intake OIDC mới là ranh giới bảo mật thực sự.
- **`intake`** là URL dịch vụ intake của hub (người vận hành cung cấp cho bạn). Khi có nó, `kb publish` đẩy tag `kb-publish/<ts>` và CI chạy `kb ci-publish`, lệnh này xác thực với intake bằng GitHub Actions OIDC ngay trong job của bạn và không cần secret nào. Khi không có, `kb publish` đi đường snapshot (mirror trực tiếp hoặc một PR trên hub, không CI, không intake).

Không bao giờ đặt token trong `hub:` hay `intake:`. File này được commit, nên token sẽ nằm lại trong lịch sử. `kb doctor` sẽ cảnh báo; hãy viết lại URL, truyền URL có credential qua biến môi trường `STRATA_KB_HUB` (nó ghi đè file), và đổi token. Mọi máy dùng URL hub có credential cần Git 2.31 trở lên.

`hub:` và `STRATA_KB_HUB_URL` là hai giá trị khác nhau: `hub:` là URL git hoặc đường dẫn của repo hub; `STRATA_KB_HUB_URL` là URL HTTP gốc của hub, dùng cho MCP và auto runner.

2.3 Kết nối trợ lý của bạn

`.mcp.json` (Claude Code) và `.cursor/mcp.json` (Cursor) trỏ tới hub qua hai chỗ giữ chỗ, `${STRATA_KB_HUB_URL}` và `${STRATA_KB_HTTP_TOKEN}`. Chạy `kb mcp-setup` (trong trợ lý: `/kb-mcp-setup`). Lệnh hỏi địa chỉ HTTP gốc và token của hub (token lấy từ người quản trị hub), ghi cả hai vào `.env`, bảo đảm git bỏ qua file đó, rồi xác minh với hub, nên URL sai và token bị từ chối cho ra hai lỗi khác nhau. Chạy lại không đối số sẽ đọc cả hai giá trị từ `.env` và xác minh lại.

Hãy dùng prompt ẩn hoặc biến `STRATA_KB_HTTP_TOKEN` thay vì `--token`. Cờ đó để token lại trong lịch sử shell và trong danh sách tiến trình mà người dùng khác trên máy đọc được. `kb mcp-setup` cũng cảnh báo khi URL là `http://` thường tới một host không phải loopback, vì token khi đó đi qua mạng dạng rõ.

Cấu hình MCP đọc hai biến đó từ môi trường của bạn, không đọc từ `.env`, đúng lúc trợ lý khởi động. Hãy nạp file (`set -a; source .env; set +a`, hoặc dùng direnv) rồi khởi động lại trợ lý.

Một phiên bản cho mọi repo. Mọi repo chạy cùng phiên bản strata-kb với hub. Hãy nâng cấp hub trước. Sau đó, khi `kb mcp-setup` đã chạy (ở trên), chạy `kb doctor` trong repo này: nó cài phiên bản của hub, đặt các pin `strata-kb==<phiên bản của hub>` trong `.github/workflows/`, và làm mới skill cùng command. Trong CI nó không bao giờ cài, đặt lại pin hay làm mới skill; một job CI chạy `kb doctor` với `STRATA_KB_HUB_URL` và `STRATA_KB_HTTP_TOKEN` được đặt sẽ báo sai lệch phiên bản như một lỗi (không workflow dựng sẵn nào chạy nó). Khi `STRATA_KB_HUB` hoặc `--hub` được đặt, `kb doctor` bỏ qua đường MCP và việc kiểm tra phiên bản, và chỉ báo rằng phiên bản chưa được kiểm tra với hub. Các file trong `.github/instructions/` và `.cursor/rules/` chỉ được làm mới qua `kb init`. Nếu `pip install strata-kb` không ghim phiên bản nằm trong một job có `id-token: write`, mỗi lần chạy sẽ lấy bất cứ bản nào PyPI đang phục vụ, trong một job có quyền ghi vào hub; đó là lý do các workflow được ghim phiên bản.

2.4 Bật cổng CI và lượt quét hằng ngày

Thiết lập	Ở đâu	Là gì
`STRATA_KB_HUB`	Actions, **Variables**	Hub mà cổng dùng để phân giải tham chiếu
`KB_HUB_TOKEN`	Actions, **Secrets**	Một token chỉ đọc, chỉ cho hub riêng tư
`KB_FAIL_ON_STALE`	**Variables**, tùy chọn	Bất kỳ giá trị nào cũng làm cho một bản sửa ở thượng nguồn khiến cổng thất bại

Sau đó, trong branch protection, **bắt buộc check `lint`** (job trong workflow `kb-ticket-lint`) trên nhánh mà ticket của bạn được merge vào. Lint chạy trong CI tự nó không chặn merge; bắt buộc check mới chặn. Workflow chạy trên mọi pull request, không chỉ khi có thay đổi dưới `tickets/` hay `missions/`, và đó là chủ ý: bộ lọc đường dẫn trên một check *bắt buộc* sẽ khiến một PR không liên quan chờ mãi một check không bao giờ bắt đầu. Job xem diff và thoát êm khi không có gì để lint.

> Một pull request mở **từ fork** không đọc được secret của repository, nên với hub riêng tư, cổng thất bại ở đó với thông báo không tới được hub. Đó là cổng từ chối báo xanh khi chưa kiểm tra, không phải lỗi mạng. Hãy merge đóng góp từ fork qua một nhánh trong chính repository này.

`kb-doctor.yml` chạy `kb ticket stale` trên `tickets/` và `missions/` mỗi ngày lúc 03:17 UTC và theo yêu cầu, với cùng variable và secret. Các workflow publish không cần cả hai: chúng xác thực bằng OIDC.

---

3. Công việc hằng ngày

Việc cần làm	Chạy	Kết quả
Lập kế hoạch cho tính năng lớn	`/ba-mission-plan`	`missions/M-<slug>.md`
Viết ticket	`/ba-ticket-author`	`tickets/<folder>/<ticket-id>.md`
Báo một lỗi	`/ba-bug-report`	`tickets/bugs/BUG-<n>.md`
Trả lời câu hỏi của Dev	`/ba-answer-questions`	ticket được amend, câu trả lời nằm trên hub
Thay đổi mission hoặc ticket	`/ba-mission-plan --amend M-<slug>`, `/ba-ticket-author --amend <ticket>`	một revision, một dòng change log
Xem story nào kế tiếp	`kb mission next`	done / ready / draft / to-draft / blocked
Publish kế hoạch	`/kb-publish`, hoặc merge vào `main` (nhu đã scaffold)	kế hoạch nằm trên hub

Mọi việc agent làm, bạn đều làm được ở terminal: `kb query "<text>" [--tags t]`, `kb get <doc> <section> [--level l2]`, `kb context new --refs "doc §4.2, doc §4.3"` (dán block vào dưới `## KB context`), `kb ticket lint <file>`, `kb mission lint <file>`.

3.1 Lập kế hoạch cho tính năng lớn: mission

Một tính năng trải qua nhiều user story cần một **mission plan** trước; việc nhỏ đi thẳng vào ticket. Chạy `/ba-mission-plan` và mô tả nhu cầu. Quy trình là **Intake, Ground, Draft, Split, Ground services, Pin, Lint, Maturity review, Review**. Nó lưu `missions/M-<slug>.md` gồm sơ đồ C4 Level 1 (System Context), sơ đồ C4 Level 2 (Container), phần chia phạm vi và một backlog user story có id suy ra từ id của mission.

```
kb mission lint missions/M-checkout-revamp.md
```

Cổng yêu cầu đúng cấu trúc, đủ hai sơ đồ, backlog đúng dạng và mọi trích dẫn phân giải được ở phiên bản đã ghim. Cảnh báo `0/N US drafted` là bình thường: ticket chưa tồn tại. Backlog đã đầy đủ chưa vẫn là phán đoán của **bạn**. Nếu bỏ một story, hãy để số của nó nghỉ hưu: đánh số lại sẽ làm hỏng tên file của các ticket đã soạn.

SA bám nguồn cho danh sách service (bước 5, chạy hộ bạn): `/sa-ticket-ground --mission` điền `## Services & order`, và cổng báo FAIL ở mỗi dòng `[NEW: D<n>]` cho tới khi bạn chuyển nó sang `DECIDED` (3.3). Trong repo skeleton, agent dừng ở bước 5b, **Decide**, và cho bạn xem bảng một lần.

3.2 Viết ticket

Chạy `/ba-ticket-author` và mô tả nhu cầu nghiệp vụ. Agent chạy quy trình chín bước; việc của bạn là bước 1, 3 và 9.

```
 1 Intake            you describe the need
2 Parent mission    optional, name the mission this story belongs to
3 Ground            kb_search; YOU pick which sections apply
4 Draft             story, Given/When/Then ACs, use cases, business flow
5 Pin               kb_context_new embeds the pinned block
6 Lint              kb ticket lint until DoR: PASS
7 Ground technical  the SA fills Technical grounding; kb ticket check
8 Maturity review   two independent reviews, one round by default
9 Review, save      you read it, commit it, paste it into the tracker
```

- **Ground.** Agent cho bạn xem **mọi section ứng viên**, không chỉ cái nó thích nhất. Hãy chọn những cái thực sự áp dụng. Phán đoán miền nghiệp vụ của bạn không thể giao đi ở đây: agent không thể biết một section đã bị một chính sách nội bộ thay thế trên thực tế.
- **Maturity review.** Hai lượt review độc lập theo `docs/review-rubric.md` (Business coverage, Dev implementability). Khoảng trống mà nó không tự lấp được trở thành một câu hỏi mở có chủ, `OPEN(<owner>)`, không bao giờ là phỏng đoán: hãy đọc chúng, đó là chỗ agent biết mình không biết. Giới hạn số vòng nằm trong skill. Điểm review do agent tự báo: `kb ticket lint` chỉ kiểm `## Review record` có đúng dạng, không kiểm điểm đạt được bằng cách nào.
- **Review và lưu.** Bản nháp nằm ở `tickets/<folder>/<ticket-id>.md`. Bạn đọc, commit và dán `kb ticket export tickets/<folder>/<ticket-id>.md` vào công cụ quản lý issue: ticket không có comment hướng dẫn, `## Definition of Ready` và `## Review record`.

Liên kết với mission. Hãy nêu mission cha khi soạn một story từ backlog của nó. Agent ghi `> Parent mission: <mission-id> @ rev <N>` dưới tiêu đề và lưu `tickets/<mission-id>/<mission-id>-US<n>.md`. Mọi story mà `- Blocked by:` của ticket nêu tên cũng phải nằm trong ô `Depends on` của nó ở `## Sequencing` của mission, nếu không `kb ticket lint` thất bại: auto runner của Dev chỉ đọc `Depends on`. `kb ticket tidy` chuyển một thư mục `tickets/` phẳng vào bố cục theo mission dựa trên dòng parent.

Một ticket, một repo đích. Một ticket được xây trong đúng một dev repo, và hub định tuyến nó tới đó: dòng `> Target repo: <repo-id>` trong khối tiêu đề (như `> Type:`, không bao giờ nằm trong phần thân) được ưu tiên; không có thì repo đích là repo của mục `Grounded on: <repo>:<repo>-code` có tiền tố repo đầu tiên, do SA ghi (mục `-code` không có tiền tố được phân giải theo repo duy nhất nắm giữ nó trên hub). `kb ticket lint` báo lỗi khi dòng tiêu đề và một grounding có tiền tố nêu hai repo khác nhau. Hub chỉ có một dev repo thì mọi ticket đi vào repo đó và không cần gì thêm. Có từ hai dev repo trở lên, ticket không có đích bị chặn với lý do `no target repo`, còn đích không phải dev repo trên hub bị chặn với `target <x> is not a dev repo on the hub`. Công việc trải trên hai repo là hai ticket, hoặc một parent story tách thành các child (3.10). Dòng `- Blocked by:` của ticket ngoài mission chỉ chặn theo các id đã publish trong cùng plan BA; mã tracker hay văn xuôi không bao giờ chặn.

3.3 SA bám nguồn cho nửa kỹ thuật

Bước 7 chạy việc này hộ bạn. Khi lint báo `DoR: PASS`, agent lưu bản nháp và chạy `/sa-ticket-ground` như một subagent riêng. Nó điền `## Technical grounding` do SA sở hữu (service, file, bảng, route, trang, job, symbol, lệnh test), mỗi mục là một id section của `<repo>-code` trên hub, và không bao giờ sửa các section của bạn. Code chưa tồn tại là một quyết định thiết kế: SA đề xuất một dòng `## Technology decisions` trong mission cha (trạng thái `OPEN`, một người sở hữu thật) và ghi `[NEW: D<n>]`. Không có mission cha thì nó ghi một ghi chú tự do `[NEW: <reason>]`.

`kb ticket check tickets/<folder>/<ticket-id>.md` chặn nó (exit `0` PASS, `1` FAIL):

Output	Ý nghĩa	Việc cần làm
PASS, `[note] ... still valid`	Không gì ticket trích dẫn thay đổi	Không cần làm gì
PASS, `[warn]` kèm diff	Một section được trích dẫn đã đổi, mọi tên được trích dẫn vẫn đúng	Developer đánh giá diff
FAIL, `re-run /sa-ticket-ground`	Một id, cột, route, lệnh hay file được trích dẫn đã biến mất	Chạy `/sa-ticket-ground` bằng tay
FAIL, nêu một dòng `OPEN` và chủ của nó	Một `[NEW: D<n>]` trỏ tới quyết định chưa chốt	Chuyển nó sang `DECIDED` là quyền của bạn, không bao giờ của agent
FAIL, `Open decisions` không rỗng	Tài liệu không chứng minh được điều gì đó	Do bạn hoặc Dev quyết; không phải re-ground

Bản bàn giao liệt kê các dòng còn mở trong block `## Needs input`. Cách SA phân biệt "chưa xây" với "chưa quét" nằm ở 6.4.

3.4 Story nào kế tiếp

> `kb mission next`

Trạng thái	Nghĩa là
`done`	Id của nó nằm trong một dòng `hist.*` của `<repo>-svc` trên hub, được ghi lúc Dev bàn giao và publish bằng `kb publish` của Dev sau khi merge
`ready`	Ticket tồn tại và mọi ô `## Definition of Ready` đã được tick
`draft`	Ticket tồn tại nhưng còn ô chưa tick, thiếu section, hoặc mission cha đã đi tiếp
`to-draft`	Chưa có ticket, mọi story trong `Depends on` đều `done`, mọi dòng D `Blocks` nó đều `DECIDED`, không câu hỏi mở nào nếu nó trong `blocks:`
`blocked`	Lý do được nêu tên: `US <id> not done`, `D<n> OPEN (owner: <x>)`, `Q<n> open (owner: <x>)`

Báo cáo kết thúc bằng `Next: <us-id> — <title>`; `/ba-ticket-author` không đối số sẽ chạy nó trước và đề xuất story đó. Một phụ thuộc mới chỉ `draft` hoặc `ready` vẫn chặn: done nghĩa là đã publish lên hub. Một ticket cũng là `draft`, lý do `parent mission rev <M> affects this story`, khi `> Revision:` của mission cao hơn pin `@ rev` của ticket **và** một dòng change log sau đó liệt kê story trong `Affected`. Lệnh chỉ đọc và thoát `0` trừ khi dùng sai lệnh. `--kb-dir` được đọc trước khi nó chứa tài liệu `-svc` (máy dev), hub đọc sau; hub không tới được hay `-svc` chưa publish chỉ thành một dòng `note:`. Với `--from-hub` (và `--repo <ba-repo-id>`) nó đọc kế hoạch đã publish thay vào đó: ticket đã publish là `ready`, và không thấy được `draft`.

3.5 Báo một lỗi

Một lỗi ở thứ đang chạy là **bug ticket**: không có "As a ... I want ...", và tiêu chí chấp nhận là "các bước không còn tái hiện kết quả sai". Chạy `/ba-bug-report`. Nó hỏi cái gì hỏng, môi trường, các bước chính xác, kết quả mong đợi và thực tế, mức độ nghiêm trọng và ảnh chụp màn hình, soạn `tickets/bugs/BUG-<n>.md` từ `docs/tickets/BUG-TEMPLATE.md` và lint cho tới khi `DoR: PASS`. Bug bỏ qua maturity review và technical grounding.

- **Dòng tiêu đề.** `> Type: bug|hotfix` (hotfix ship trước bản phát hành kế tiếp) và `> Severity: S1..S4`, cả hai bắt buộc; tùy chọn `> Related ticket: <id|none>`. Không có `> Parent mission:`. SA đề xuất bản sửa nằm ở dev repo nào và agent ghi `> Target repo: <repo-id>` (bạn chấp nhận hoặc đổi; hub chỉ có một dev repo thì có thể bỏ dòng này, hub sẽ định tuyến bug tới repo đó). Không có đích, hub có từ hai dev repo trở lên sẽ chặn bug với lý do `no target repo`.
- **Ảnh chụp.** Chỉ PNG hoặc WebP, trong `tickets/bugs/BUG-<n>/`, tham chiếu dạng `![caption](BUG-<n>/<file>.png)` dưới `## Attachments`; không có thì `N/A — <reason>`.
- **Phần của bạn.** Tick Definition of Ready sau khi đọc bản nháp (agent không bao giờ tick), commit bug cùng thư mục của nó và publish. Sau đó Dev chạy `/dev-fix-bug BUG-<n>`.

Bug trải trên nhiều repo

Một bug có thể cần sửa ở hai dev repo. Khi đó nó thành **parent bug**, mỗi repo có một child bug.

- **Repo phản hồi đầu tiên.** SA đọc bằng chứng trong code và đề xuất repo nào xem bug trước. Agent ghi vào `> Target repo:`. Chỉ Dev của repo đó thấy `/dev-fix-bug BUG-<n>`.
- **Đề xuất tách.** Dev thấy nguyên nhân gốc nằm cả ở repo khác. Dev viết `## Child bug proposal` trong `rootcause.md`, hỏi `Split proposal for BUG-<n>: <một dòng>` và lượt chạy kết thúc `blocked-split BUG-<n>`. Claim của Dev được dừng lại, không mất.
- **Bạn trả lời.** Trong `/ba-answer-questions`, đề xuất nằm ở mục `### Split proposals`. Muốn chấp nhận, chạy `/ba-bug-report --child-of BUG-<n>`. Lệnh này viết lại `BUG-<n>` thành `> Kind: parent bug` có bảng `## Split` và tạo các file child `BUG-<n>.<repo>`. Bạn tick DoR của từng child. Sau đó agent trả lời với revision mới và id các child. Agent ghi câu trả lời trước. Chỉ sau đó bạn mới commit các file và chạy `kb publish`. Câu hỏi chuyển sang answered khi các child đã lên hub. Publish trước khi có câu trả lời thì bị từ chối ("hub already has revision N+1"); không publish thì câu hỏi vẫn ở waiting. Muốn từ chối, trả lời mà không đổi gì.
- **`Own fix`.** Dòng riêng của parent ghi `yes` khi repo đó cũng tự sửa một phần. Cột `Blocked by` của dòng này cho biết nó chờ child nào.
- **Xong.** Parent bug xong khi mọi child xong và, nếu `Own fix: yes`, phần tự sửa cũng xong. Hub hiển thị `k/N done` và `own fix done|pending`.

Mẫu: `docs/tickets/PARENT-BUG-TEMPLATE.md`.

3.6 Trả lời câu hỏi của Dev

Khi thiết kế hay lập kế hoạch cho một ticket, Dev có thể hỏi một câu hỏi nghiệp vụ quyết định ý nghĩa của một tiêu chí chấp nhận (phía Dev). Các câu hỏi nằm trên hub. Chạy `/ba-answer-questions [ticket]`: nó liệt kê các câu hỏi đang mở của repo này và đi qua từng ticket một, cũ nhất trước, mỗi câu kèm bối cảnh, các AC nó chặn và người

hỏi. Câu trả lời là của bạn; agent không bao giờ viết hộ. Nó cần MCP trỏ tới server HTTP của hub (`kb mcp-setup`); không có CLI cho việc này.

- **Định dạng.** Câu hỏi và câu trả lời dùng nhãn tiếng Anh cố định (`**Decision needed:**`, `**Decision:**`, `**Ticket changes (rev N):**`, và các nhãn khác); skill tự ghi chúng, còn nội dung theo ngôn ngữ của ticket.
- **Một lần amend cho mỗi ticket.** Câu trả lời làm đổi hành vi sẽ amend ticket như ở 3.7: `> Revision:` thành N+1, một dòng `## Change log` với Trigger `question`, What changed `Q<ids>: <one line>`, một vòng maturity review, và lint đạt. Mọi câu hỏi của một ticket gộp vào cùng một lần amend.
- **Tick lại Definition of Ready.** Lần amend bỏ tick các ô có bằng chứng đã đổi, và `kb plan-ingest` loại ticket có ô chưa tick đủ, khiến nó rời khỏi hub. Cho tới khi mọi ô được tick, skill không ghi câu trả lời cho ticket đó.
- **Chờ publish.** Skill không bao giờ commit, push hay publish. Cho tới khi revision N+1 tới hub, câu hỏi hiện là *waiting for publish* và Dev được báo đừng hành động theo nó; một câu trả lời không đổi gì được ghi là đã trả lời ngay. Menu **Q&A** của hub (`/ui/questions`) cho thấy câu hỏi đang mở, đang chờ và đã trả lời.

3.7 Giữ mission và ticket luôn mới

Mọi mission và ticket mang `> Revision: <N>` dưới tiêu đề, bắt đầu từ 1, và một `## Change log` chỉ-thêm-vào đặt trước `## Review record`:

```
| Rev | Date | Trigger | What changed | Affected | Review |
|---|---|---|---|---|---|
| 2 | 2026-10-14 | kb-context, scope | Refund window cut to 14 days | M-checkout-US3, M-checkout-US5 |
```

`Trigger` là một hoặc nhiều giá trị `kb-context`, `architecture`, `scope`, `delivery`; `question` chỉ do `/ba-answer-questions` thêm. Trong mission, `Affected` liệt kê id story **đầy đủ** (`M-<slug>-US<n>`; `US3` trơn không khớp). Lint cần đúng một dòng cho mỗi revision `2..N`, theo thứ tự, với trigger hợp lệ.

```
/ba-mission-plan --amend M-checkout
/ba-ticket-author --amend M-checkout-US3
```

Cả hai không bắt đầu từ Intake và không đổi tên gì. Agent hỏi bạn trigger và điều gì đã đổi (phát hiện độ lệch điền sẵn câu trả lời, nó không bao giờ đoán), áp dụng các quy tắc thay đổi, tăng `Revision`, thêm dòng, lint, chạy một vòng maturity review cho các section đã đổi và đưa bạn bản diff. Nó không bao giờ commit hay push. Amend ticket sẽ **bỏ tick** các ô Definition-of-Ready có bằng chứng đã đổi.

Ai tick Definition of Ready. Agent tự tick, kèm bằng chứng, mọi ô mà một công cụ chứng minh được sau lần sửa cuối của nó (`DoR: PASS`, `kb_ticket_stale` sạch, `kb_ticket_check` PASS, tối đa 10 AC). Các ô phán đoán ("mọi AC đều là Given/When/Then và kiểm thử được", "backlog đã review với nhóm") vẫn là của bạn, và agent không bao giờ tick một câu hỏi mở.

Quy tắc thay đổi.

- **Story đã done bị đóng băng.** Story `done` giữ nguyên dòng backlog. Việc mới hoặc thay đổi là **một dòng mới với số tự do kế tiếp**; số không bao giờ được dùng lại hay đánh số lại.
- **Story chưa done có thể sửa.** Liệt kê từng cái trong `Affected` của dòng.
- **Quyết định được thay thế, không được sửa.** Không bao giờ viết lại một dòng `DECIDED`. Thêm `D<n>` với `Supersedes D<k>` trong ô Decision và đặt `D<k>` thành `SUPERSEDED`; `kb ticket check` khi đó từ chối mọi `[NEW: D<k>]`. File nào còn trích dẫn dòng cũ cần trigger `architecture`.
- **Ticket đi theo mission của nó.** Ticket có story được một lần amend mission liệt kê trong `Affected` sẽ thành `draft` và rời khỏi kế hoạch đã publish cho tới khi bạn amend nó.

3.8 Publish kế hoạch

Đưa mission hay ticket đã review lên hub để developer, SA và các nhóm khác thấy cái đã lên kế hoạch, không chỉ cái đang tồn tại. Bạn publish bằng `/kb-publish`, hoặc bằng cách merge một thay đổi của `missions/` hay `tickets/` vào `main`.

```
/kb-publish          # or: kb publish
```

`kb publish` chạy `kb plan-ingest` và `kb build` trước, nên cái bạn review ở máy là cái CI gửi đi. Sau một lần bạn đồng ý rõ ràng, nó gắn tag `kb-publish/<timestamp>` lên commit và đẩy tag. Tag kích hoạt `kb-publish.yml` (`kb plan-ingest`, `kb build`, `kb ci-publish`) và intake mở một pull request trên hub; nội dung sống khi PR đó được merge (ngay lập tức nếu Registry của hub bật auto-merge cho repo của bạn, nếu không thì khi người quản trị hub merge nó). Intake chỉ chấp nhận `refs/tags/kb-publish/*`. Trên đường intake, `kb publish` từ chối khi `.kb/`, `missions/` hoặc `tickets/` có thay đổi chưa commit, vì CI publish commit đã gắn tag. `.kb/<repo_id>-plan/` và `.kb/index.yaml` được sinh tự động và git bỏ qua: không bao giờ commit chúng.

Merge cũng là publish. Một push lên `main` chạm tới `missions/` hoặc `tickets/` sẽ chạy `kb-publish-request.yml`, workflow gắn tag lên commit vừa merge và kích hoạt workflow publish (chạy tay để publish lại khi không có thay đổi; với nhánh mặc định khác, đổi `main` ở hai chỗ trong file đó). Giống `kb publish`, việc này cần `intake:` đã đặt, vì `kb-publish.yml` chạy `kb ci-publish`.

`kb plan-ingest` quyết định cái gì được ship, offline và không dùng LLM. Hãy chạy nó trước để xem:

Mục	Được publish khi
Mission	`kb mission lint` không thấy lỗi cấu trúc; Definition of Ready của nó không phải là cổng
Ticket	`kb ticket lint` không thấy lỗi cấu trúc, **và** mọi ô DoR đã tick, **và** `kb mission next` sẽ không hạ nó xuống `draft`

Trích dẫn còn phân giải được hay không không phải cổng publish (cổng PR kiểm tra việc đó), và bản nháp không bao giờ được ship. Mỗi mục bị bỏ lại in một dòng và không bao giờ làm lệnh thất bại:

```
skipped: tickets/M-checkout/M-checkout-US3.md — Definition of Ready not fully ticked (6/8)
```

Các lý do chính: `lint error: <first error>` (chạy `kb mission lint` hoặc `kb ticket lint` trên file); `Definition of Ready not fully ticked (<ticked>/<total>)` (hoàn tất review rồi tick); `parent mission rev <M> affects this story` (amend ticket, 3.7). Với lỗi hình thức file, thông báo sẽ giải thích.

Một mục không còn đủ điều kiện sẽ rời đi ở lần publish kế tiếp; lịch sử git của hub giữ các phiên bản trước. `kb plan-ingest` chỉ thoát `1` khi nó không chạy được, không bao giờ vì một mục bị bỏ qua.

Trên hub. Kế hoạch là một tài liệu, `<repo_id>-plan`, cạnh `-code` và `-svc`: mỗi mission một nhóm cùng các ticket đã publish của nó, cộng một nhóm `tickets` cho ticket không có mission đã publish. Agent tìm nó bằng `kb_search`; con người đọc nó ở menu **Ticket** của web UI (`/ui/tickets`: mọi repo BA, mission của chúng và số lượng theo trạng thái; đi sâu vào backlog của một mission và vào từng ticket).

Trạng thái được suy ra và chỉ đọc: **done**, **ready**, **not ready** (chưa có ticket đã publish) hoặc **blocked**. Muốn đổi, hãy đổi nguồn. Công cụ quản lý issue, không phải hub, là nơi cho "đang làm" và người được giao. Một Dev chỉ có id ticket sẽ lấy nó từ đây (Hướng dẫn cho Developer).

3.9 Giữ trích dẫn luôn mới

Một ticket viết hồi tháng Ba dựa trên tài liệu được sửa hồi tháng Năm không được lặng lẽ bắt đầu mang nghĩa khác.

```
kb resolve tickets/ABC-123.md          # ok | stale | broken, per ref
kb diff hr-handbook --against <rev>    # what actually changed
```

Trạng thái	Ý nghĩa	Việc cần làm
`ok`	Nội dung đã ghim vẫn khớp với cái đang publish	Không cần làm gì
`stale`	Nguồn đã được sửa sau khi bạn trích dẫn	`kb diff`; quyết định xem AC có cần cập nhật không
`broken`	Tham chiếu không còn phân giải được	Bám nguồn lại và ghim lại bằng `kb_context_new` hoặc `kb context new`

Phần lớn độ cũ là vô hại, như sửa lỗi chính tả ở thượng nguồn; một phần là yêu cầu không còn khớp thực tế, và phán đoán là của bạn. `kb ticket lint <file> --fail-on-stale` (hoặc biến `KB_FAIL_ON_STALE` cho cổng CI) làm độ cũ gây thất bại; `kb ticket stale [paths]` quét toàn bộ. Cổng PR không bao giờ nhìn lại một ticket đã merge, nên `kb-doctor.yml` quét hằng ngày và, khi còn thứ gì cũ hoặc hỏng, giữ **một** issue mở gắn nhãn `kb-stale-citations` (đóng khi mọi thứ đã mới):

Loại	Ý nghĩa	Việc cần làm
`kb-context`	Một section file trích dẫn trong `## KB context` đã đổi hoặc không còn phân giải được	`kb diff`, rồi amend với `kb-context`, trigger này ghim lại
`architecture`	Một service, bảng, route, integration hay lệnh mà file trích dẫn trong `## Services & order` hoặc `## Technical grounding` đã đổi trong `-code` của repo Dev kể từ lúc bám nguồn, hoặc đã bị xóa	Amend với `architecture` để `sa-ticket-ground` bám nguồn lại; kiểm tra các AC dựa vào nó
`architecture`, `baseline not on hub`	Revision `Grounded on:` không có trên hub: PR của Dev mang nó chưa bao giờ được merge hoặc đã bị thay thế	Chờ `-code` của repo Dev được publish, rồi amend với `architecture`

Chỉ các section mà file **trích dẫn** mới được so sánh, và không phải dòng nào cũng cần amend: nếu yêu cầu vẫn đúng, hãy amend với pin mới và nói rõ điều đó trong dòng change log.

3.10 Story trải trên nhiều repo

Một story cần việc ở nhiều dev repo là một **parent story** được tách thành một **child ticket** cho mỗi repo. Parent là story nghiệp vụ; mỗi child là một ticket bình thường, được xây trong đúng một repo. Chỉ viết parent sau khi hub được nâng lên bản có parent story (xem Nâng cấp trong Hướng dẫn Hub, 3.8): hub cũ không biết định dạng này.

Parent. `kb init` ghi mẫu vào `docs/tickets/PARENT-TEMPLATE.md`. File là `tickets/<folder>/<US-id>.md` với `> Kind: parent story` và `> Revision: <N>` trong khối tiêu đề, rồi `## User Story`, `## Split` và `## Change log`

(cùng sáu cột như của ticket). Parent không có `## Acceptance Criteria` và không có `## Definition of Ready`: mỗi AC nằm trong đúng một child.

```
# Pay by card US-12
> Kind: parent story
> Revision: 1

## User Story
As a shopper, I want to pay by card, so that I can finish my order.

## Split
| Child | Repo | AC | Blocked by | Own fix | Why |
|---|---|---|---|---|---|
| US-12.payments-api | payments-api | AC1, AC2 | - | - | charges the card |
| US-12.web-app | web-app | AC3, AC4 | US-12.payments-api | - | the checkout form |

## Change log
| Rev | Date | Trigger | What changed | Affected | Review |
|---|---|---|---|---|---|
|  |  |  |  |  |  |
```

Khối tiêu đề của child `tickets/<folder>/US-12.web-app.md`:

```
# Pay by card, checkout form US-12.web-app
> Revision: 1
> Parent story: US-12 @ rev 1
> Target repo: web-app
```

Phần còn lại của child là một ticket bình thường (AC, DoR và grounding riêng); `docs/tickets/PARENT-TEMPLATE.md` có parent đầy đủ.

Các child. Id của child là `<parent-id>.<repo-id>` (`US-12.payments-api`, `US-12.web-app`); child sau cho cùng một repo là `<parent-id>.<repo-id>.2`, rồi `.3`. Id repo có dấu chấm không thể là đích của split. Child lặp lại `## User Story` của parent từng chữ một và mang hai dòng tiêu đề: `> Parent story: US-12 @ rev 1` và `> Target repo: payments-api`. Child của parent thuộc một mission không có dòng `> Parent mission:`; nó kế thừa từ parent. Một AC thay thế AC trong child đã xong được ghi là `AC7 (replaces AC2)` trong split. `Blocked by` trong split nêu id child của cùng bảng; `- Blocked by:` của chính child chỉ tính các id đúng như vậy.

Sửa parent. Tăng `> Revision:` và thêm một dòng `## Change log` (chỉ các trigger hiện có) nêu các child bị ảnh hưởng, trong cùng một lần sửa. Child vẫn ghi `@ rev <cũ>` và được nêu trong một dòng sau đó bị hạ cấp và rời khỏi hub cho đến khi bạn amend. Child cũng rời khỏi hub khi mission của parent amend parent, cho đến khi bạn amend parent.

Lint kiểm tra gì. `kb ticket lint` trên parent kiểm tra split (cột, mỗi AC có đúng một chủ, id `Blocked by` có trong split, không có vòng) và revision. Trên child, nó kiểm tra tiêu đề so với parent trong cùng thư mục. Id ticket trùng ở bất kỳ đâu dưới `tickets/`, hoặc id ticket trùng id mission, là lỗi. Với `--hub`, một id đã được repo BA khác publish, hoặc parent đã xong trong `-svc` của một dev repo, cũng là lỗi. Cổng CI còn lint từng parent trong mọi thư mục mà PR chạm tới, nên xóa một child làm parent thất bại. `kb ticket lint` kiểm tra parent bug theo cùng cách (xem "Bug trải trên nhiều repo" ở 3.5).

Publish thất bại là không publish gì. `kb plan-ingest` và `kb publish` publish parent mà không cần DoR. Khi bất kỳ parent nào không qua lint, không có gì được publish và hub giữ kế hoạch tốt gần nhất. Chạy `kb ticket lint` trên parent và sửa lỗi đầu tiên.

Trạng thái. Hub hiển thị parent là `in progress`, `blocked` hoặc `done`, kèm `k/N done` (cộng các blocker còn mở của mission). Trang của nó liệt kê các child cùng repo và trạng thái, và liên kết từng child đã viết; trang của child liên kết tới parent và các child anh em. Child chờ các id `Blocked by` trong split; child chưa viết hiển thị `to-draft` (hoặc `blocked` khi mission chặn nó). **Runner không bao giờ nhận parent**, và claim trên parent trả 400 (`parent_story`): runner của Dev làm các child. Child chưa viết được hub liệt kê là to-draft; `/ba-ticket-author`

`<parent-id>.<repo-id>` soạn nó.

Split được đề xuất thế nào. Bạn không phải tự quyết split một mình. Trong `/ba-ticket-author`, bước 1b "Scope by repo" đọc xem nhu cầu chạm những repo nào (`sa-ticket-ground --scope "<need>"`) rồi đề xuất một story hoặc một parent với một child cho mỗi repo, kèm AC của từng child. Chấp nhận thì skill viết parent và các child, và ghi người chấp nhận dưới `## Change log` của parent bằng `**Split decision:** <YYYY-MM-DD> — <n> children: <ids> — evidence: <citations> — accepted by: <who>`. Từ chối thì nhu cầu vẫn là một ticket (không có parent nào được viết). Muốn soạn một child sau, chạy `/ba-ticket-author <parent-id>.<repo-id>`: skill đọc parent và chỉ viết child đó, không hỏi lại split.

`/ba-mission-plan` (bước 5c, "Repos per story") có thể đề xuất cùng split ấy ngay khi lập kế hoạch mission. Parent đó được viết với `-` ở cột AC của mọi dòng split và không có file child; `kb ticket lint` chấp nhận nó, hub liệt kê mỗi child là to-draft, và picker soạn từng child một. Khi viết một child, điền AC của nó vào dòng split. Ở chế độ full-auto, `/ba-mission-plan` không bao giờ tách: việc tách xảy ra lúc soạn ticket, ở bước 1b, và một agent cùng một reviewer có ngữ cảnh mới quyết định chấp nhận hay từ chối.

Lint đưa ra hai cảnh báo hỗ trợ việc này. `kb ticket lint` cảnh báo khi parent thiếu dòng `**Split decision:**` dưới `## Change log`. Nó cũng cảnh báo khi một ticket đơn có grounding trên từ hai repo trở lên: `grounding touches several repos (<a>, <b>) — split it into a parent story (BA Guide 3.10) or keep one ticket`. Mission có `## Services & order` nêu nhiều repo được kiểm tra với tài liệu `-code` của từng repo: viết id dạng `<repo>:svc.<name>`.

Đổi story ở cấp parent. Chạy `/ba-ticket-author --amend <parent-id>`. SA đề xuất thay đổi: child nào cần amend (và AC nào của chúng), child nào thêm mới, child nào giữ nguyên. Bạn chấp nhận, sửa hoặc từ chối. Khi chấp nhận, parent lên rev N+1, các dòng split được giữ đồng bộ và một dòng change log nêu mọi child được amend và child mới. AC mới lấy số trống kế tiếp trong cả split; AC thay thế một AC của child đã xong được viết `AC<n> (replaces AC<m>)`. Child đã done không bao giờ bị amend: thay đổi trở thành child mới `<parent-id>.<repo-id>.2` (rồi `.3`). Parent đã done thì bị đóng băng, thay đổi tiếp theo là một story mới. Đổi `## User Story` ảnh hưởng mọi child; nếu có child đã done thì đó là một story mới. `kb ticket lint` báo lỗi khi một dòng change log của parent (rev 2 trở lên) ghi trong `Affected` thứ không phải child trong split.

Câu hỏi về một child, ghi ở parent. Dev hỏi về `US-12.web-app` được ghi vào `US-12`, với child là nguồn. Danh sách Q&A của mọi child gồm cả câu hỏi của parent, nên Dev của `US-12.payments-api` cũng thấy. `/ba-answer-questions` xử lý một câu hỏi của parent đúng một lần, ở parent. Mỗi dòng ghi `blocks this ticket: yes` hoặc `no`. Cổng của Dev chỉ dừng khi là `yes`.

Chặn được đề xuất và được xác nhận. Dev chặn ngay các AC của ticket mình. Câu hỏi có thể chặn child *anh em* chỉ được *đề xuất* (`kb_question_propose_blocks`, kèm lý do). Nó không chặn ai cho tới khi bạn xác nhận (`kb_question_confirm_blocks`). Đề xuất mới xóa xác nhận trước đó. Chỉ id AC được tính; phần lý do không chặn gì. Tên trong Q&A do người dùng tự khai, nên `confirmed_by` là dấu vết kiểm toán, không phải kiểm soát truy cập.

Khi nào câu hỏi ở parent thành answered. Trả lời một lần, nêu các child bị ảnh hưởng làm ticket sections. Câu hỏi giữ trạng thái *waiting for publish* cho tới khi parent lên hub ở rev mới **và** mọi child được nêu có trên hub với pin `@ rev N+1` trở lên. Câu trả lời không đổi ticket nào (không rev, không child) được ghi là đã trả lời ngay.

---

4. Các chế độ tự động hóa

Chế độ full-auto (tùy chọn). Một công tắc cho mỗi repo `ba` và `dev`, cho phép hai repo tự chạy, trên máy của bạn, cho tới khi mọi ticket trên hub xong. Không bước nào chờ con người: agent tự quyết từng dòng D và câu hỏi mở, trả lời câu hỏi của Dev, tick Definition of Ready sau lần review của chính nó, merge rồi publish. Nó ghi lý do vào một dòng `**Auto decision:**` dưới `## Change log`. Nó được xây cho **một repo BA và một repo Dev trên mỗi hub**; `kb auto on` cảnh báo khi registry liệt kê nhiều hơn một repo Dev. Nó chỉ publish kế hoạch và không bao giờ sửa cơ sở tri thức. Khi công tắc tắt thì không có gì thay đổi. Phía Dev nằm trong Hướng dẫn cho Developer.

4.1 Bảo mật: đọc phần này trước

Danh sách từ chối quyền mà `kb init` ghi ra (6.5) chỉ là phòng thủ nhiều lớp, không phải ranh giới. Một lượt chạy không người giám sát tự merge pull request của chính nó và publish lên hub mà không có ai review. Một ticket mà agent hiểu sai sẽ đi thẳng vào `main` và vào hub. `kb auto on` tự in cảnh báo này: "Auto mode merges pull requests as admin, with no review. Turn it on only for a repo where that is acceptable."

Nó merge bằng `kb pr merge` (GitHub REST), dùng quyền của tài khoản `gh` của bạn; `--admin` được chấp nhận cho tương thích với `gh` và không đổi gì. Nếu tài khoản đó là admin của repo và branch protection không bao gồm administrators, các lần merge đó vượt qua protection. Vì vậy, trước khi chạy, chủ repo **phải** bật GitHub branch protection trên `main`:

- chặn force push và xóa nhánh;
- **bao gồm administrators**;
- giữ check CI là **required status check**: job `lint` của `kb-ticket-lint.yml`. Branch protection chọn check theo tên job. Nếu không có check bắt buộc, "green CI" chỉ là quy ước mà agent tự tuân theo;
- **đừng** bắt buộc review chấp thuận: agent không thể tự approve PR của mình, và merge khi đó bị từ chối. Nếu bạn vẫn bắt buộc review, hãy loại administrators khỏi quy tắc đó và chấp nhận rằng agent vượt qua nó trên `main`.

Danh sách từ chối cũng không nhìn thấy nhánh hiện tại: working tree của agent không bao giờ được đứng trên `main` khi đã có commit. Các skill tạo nhánh trước; runner không canh việc này. Runner không bao giờ dùng `--dangerously-skip-permissions` hay `bypassPermissions`. Các skill hiện giả định nhánh mặc định là `main`.

4.2 Thiết lập một lần

1. Tự viết mission đầu tiên (`/ba-mission-plan`). Vòng lặp xử lý backlog nó được giao.
2. `intake:` đã đặt trong `.kb/config.yaml`, và trang Registry của hub có **auto-merge bật** cho repo này và cho repo Dev.
3. Chạy lại `kb init --kind ba`. Nó gộp allowlist quyền vào `.claude/settings.json` (các mục của bạn được giữ) và làm mới skill.
4. Export `STRATA_KB_HUB_URL` và `STRATA_KB_HTTP_TOKEN` (các giá trị phía sau `.mcp.json`), và có CLI `claude` trong `PATH`, **đăng nhập bằng gói đăng ký Claude**. Runner không bao giờ dùng API key và kế thừa bất cứ đăng nhập nào `claude` đang có.

4.3 Chạy và dừng

Việc cần làm	Lệnh	Kết quả
Bật công tắc	`kb auto on`	in cảnh báo, bật `auto.enabled`
Commit nó	commit và push `.kb/config.yaml` và `.claude/settings.json` lên nhánh mặc định	`kb auto run` từ chối cho tới khi bạn làm: worktree Q&A được checkout từ nhánh mặc định
Khởi động vòng lặp	`kb auto run` (`--once`, `--model`, `--effort`), hoặc `/kb-auto run`	`/kb-auto run` làm cả ba bước và khởi động vòng lặp tách rời; log ở `.kb/auto/run.log`
Xem đang ở đâu	`kb auto status [--json]`, `/kb-auto status`	công tắc, task, số lần thử, câu hỏi đang chờ, lý do dừng
Dừng nó	`kb auto off`, `/kb-auto off`	dừng sau task hiện tại
Chờ kết quả	`kb auto wait [--max 6600] [--interval 30]`	một dòng, dùng cho watcher trên cloud

`kb auto run` từ chối, và in mọi lý do, khi công tắc tắt, `.kb/config.yaml` hoặc `.claude/settings.json` chưa được theo dõi hay đang dirty, `auto.enabled: true` thiếu trong bản config của nhánh mặc định, `intake:` rỗng, hoặc registry của hub không đọc được, không liệt kê repo này, hay thiếu `auto_merge: true` cho nó. Mã thoát: `0` xong, `2` bị dừng, `1` bị từ chối hoặc lỗi, `130` bị ngắt.

4.4 Nó làm gì và khi nào dừng

Một tiến trình `claude -p` mới cho mỗi task. Theo thứ tự: một story bị giữ bởi dòng D đang mở hoặc câu hỏi mở đang chặn (`/ba-mission-plan --amend`), một ticket `draft` (`/ba-ticket-author --amend`), story `to-draft` kế tiếp (`/ba-ticket-author`), nếu không thì nó chờ `idle_interval` giây. Mỗi thay đổi đi qua một nhánh, một PR, CI xanh và một lần merge, và lần merge đó publish (`kb-publish-request.yml`, 3.8). Một tiến trình thứ hai, trong worktree riêng `.kb/auto/worktrees/qa`, trả lời các câu hỏi đang mở của Dev bằng `/ba-answer-questions <ticket-id>` mỗi `qa_interval` giây. Ticket và mission của nó đi cùng nhau: một lần amend mission soạn lại các ticket chưa done mà nó liệt kê trong `Affected`.

Nó dừng sau hai lần thử thất bại ở một task (CI đỏ sau hai vòng sửa thì dừng ngay); `kb auto status` cho thấy lý do, và `kb auto run` tiếp tục sau khi bạn sửa. Một ticket không tick lại được Definition of Ready thì bị đỗ (parked), không phải thất bại (bản nháp của nó đi vào `auto/qa-parked/<ticket>`); khi chỉ còn ticket bị đỗ, nó dừng với `parked: <tickets>`. Một task vẫn bị chọn lại sau 5 lần kiểm tra thì bị bỏ qua, và khi mọi task còn lại đều bị bỏ qua, nó dừng. Một lần chạy `kb-publish` thất bại được chạy lại một lần; khi lần chạy lại cũng thất bại, nó dừng với `publish failed for <tag>: <step>: <error>`. Chờ do chạm giới hạn sử dụng (15, 30, rồi 60 phút) không phải thất bại. Nó kết thúc, exit `0`, khi mọi story, ticket không có mission cha và bug đều `done` và không còn câu hỏi mở. Trạng thái và log nằm ở `.kb/auto/` (git bỏ qua).

Một tiến trình task mang `STRATA_KB_AUTO_TASK=1`, trong đó `kb auto on`, `off` và `run` từ chối (`status` vẫn chạy), nên task không thể bật công tắc cũng không thể khởi động vòng lặp thứ hai.

Prompt của runner yêu cầu mỗi task kết thúc câu trả lời bằng một dòng: `AUTO-RESULT: done`, `AUTO-RESULT: blocked-qa Q<id>[,Q<id>...]` hoặc `AUTO-RESULT: failed <reason>`, và không có gì sau nó. Dòng thiếu hoặc sai dạng được tính là một lần thử thất bại (`no result line`, `bad result line`). Skill bạn tự viết cho runner phải kết thúc theo cùng cách.

4.5 Claude Code cloud

`/kb-auto run` cũng chạy được trong một phiên Claude Code cloud (nhận diện qua `CLAUDE_CODE_REMOTE`; `STRATA_KB_CLOUD=1|0` ép bật/tắt). Hãy đặt `STRATA_KB_HUB_URL` và `STRATA_KB_HTTP_TOKEN` trong môi trường cloud. Hub được đọc trực tiếp qua API và không clone gì; các lệnh `kb` khác cần đọc hub từ chối và chỉ sang các công cụ MCP `strata-kb`. `kb pr` nói chuyện với GitHub REST API thay cho `gh pr`, vì GraphQL bị chặn ở đó. Vì không ai bấm được "trust this folder", `kb auto run` ở đó đánh dấu thư mục gốc repo và worktree Q&A là đã tin cậy trong `~/.claude.json`; các khóa khác được giữ nguyên và file không phân tích được thì bị bỏ qua.

Nhánh của phiên không push được lên nhánh mặc định. Khi công tắc của nhánh mặc định đang tắt, `/kb-auto run` mở một PR `auto/switch` chỉ bật `auto.enabled`, chờ check xanh rồi **tự merge PR đó, báo bạn trước**, bằng `kb pr merge auto/switch --squash --admin --delete-branch`. Các thiết lập auto-mode phải đã được commit trên nhánh mặc định. Lời khuyên branch protection giống như ở 4.1 áp dụng: lần merge đó cũng không có review. `/kb-auto run` chạy lại được: nó tự gỡ worktree và nhánh `auto/switch` cục bộ còn sót. PR chuyển công tắc bị bỏ mở sau một lượt chạy thất bại (check đỏ, lỗi merge) và nhánh `auto/switch` từ xa của nó phải được đóng và xóa trên GitHub bằng tay trước; skill dừng lại và nói rõ, và không bao giờ force-push. **Phiên phải được giữ mở** trong suốt lượt chạy. Chưa được kiểm chứng trên cloud thật: phiên kéo dài hơn ba giờ, push các nhánh `auto/*`, merge vượt qua một review bắt buộc, và `gh run` qua REST.

---

5. Xử lý sự cố

Triệu chứng	Nguyên nhân	Cách sửa
Lint: "ref does not resolve"	Id section sai, hoặc nội dung đó chưa được publish	Chạy lại `kb query` và ghim lại; kiểm tra PR trên hub đã thực sự merge
Lint: "kb-context tag " is not published by any document on the hub federation"	Một tag trong block `## KB context` không trỏ tới document nào đã publish. Tag được suy ra từ các document mà ref của bạn ghim; bạn không tự đặt	Xóa tag khỏi block, hoặc sửa lỗi chính tả mà thông báo gợi ý; `kb tags` liệt kê các tag thật. Đừng chạy lại `kb context new` trên ticket đã có block: nó sẽ ghi đè phiên bản đã ghim
Lint: "citation not backed by a pinned ref"	Thân ticket trích dẫn thứ mà block KB context không ghim	Thêm ref qua `kb_context_new`, hoặc bỏ trích dẫn
Lint: "missing required heading"	Thiếu một section bắt buộc, hoặc nó chỉ nằm trong khối code có rào	Đưa heading thật ra ngoài khối code
Lint: lỗi "change log" sau một lần amend	`Revision` được tăng mà không có dòng tương ứng, hoặc `Trigger` viết sai	Mỗi revision `2..N` một dòng, theo thứ tự, với năm giá trị trigger
Lint đạt ở máy, thất bại trong CI	`STRATA_KB_HUB` chưa đặt trong Actions variables, hoặc hub riêng tư mà thiếu `KB_HUB_TOKEN`	Cấu hình cả hai (2.4)
`kb doctor` cảnh báo `.kb/config.yaml` được theo dõi chứa credential trong `hub:` hay `intake:`	Một token đã bị commit	Viết lại URL không có nó, truyền qua `STRATA_KB_HUB`, đổi token (nó nằm trong lịch sử git)
`kb doctor`: "every repo on a hub uses the hub's version"	strata-kb đã cài khác với của hub	Nâng cấp hub, rồi chạy `kb doctor` ở máy khi `STRATA_KB_HUB` chưa đặt. Nó không bao giờ cài trong CI
`kb doctor`: ".github/workflows/ pins strata-kb==X but the hub runs Y"	Một pin CI khác với của hub	Chính `kb doctor` chạy ở máy sẽ ghi lại các pin; hãy commit chúng. Trong CI nó chỉ báo
Trợ lý không tới được hub	`STRATA_KB_HUB_URL` hoặc `STRATA_KB_HTTP_TOKEN` chưa đặt hoặc sai	Chạy lại `kb mcp-setup` (`/kb-mcp-setup`); nó cho biết cái nào
`kb mcp-setup`: "token was rejected" sau khi bạn nhận token mới	Chạy lại không đối số đọc token *cũ* từ `.env`	`STRATA_KB_HTTP_TOKEN=<new-token> kb mcp-setup`, hoặc xóa dòng đó khỏi `.env` rồi chạy lại
`Q&A needs the shared hub server — point MCP at it (kb mcp-setup)`	MCP trỏ tới một bản clone cache cục bộ, một server stdio hoặc bất kỳ đích nào khác ngoài server HTTP của hub; skill dừng mà không đổi ticket nào	Chạy `kb mcp-setup` và khởi động lại trợ lý
`kb query` không tìm thấy gì	Tài liệu chưa được publish, hoặc tag quá hẹp	Bỏ `--tags`; hỏi chủ hub xem PR publish đã merge chưa
`kb publish` không gắn tag hay không khởi động CI	`intake:` rỗng nên nó đi đường snapshot	Đặt `intake:` (2.2)
`kb publish`: "`.kb/` has uncommitted changes" (hoặc `missions/`, `tickets/`)	Có gì đó khác với `HEAD`, thường là `.kb/index.yaml` hay `.kb/<repo_id>-plan/` đang được theo dõi	Commit chỉnh sửa của bạn; `git rm --cached` các file sinh tự động rồi chạy lại `kb init --kind ba`
`kb publish` không đẩy được tag (phiên cloud, remote chỉ nhận nhánh), hoặc một lần merge không publish gì	Chỉ được push nhánh; hoặc `kb-publish-request.yml` chỉ chạy với push lên `main` chạm tới `missions/` hay `tickets/`	Merge vào `main`, hoặc kích hoạt workflow bằng tay; với nhánh mặc định khác, đổi `main` ở hai chỗ (3.8)
Intake từ chối lần publish của bạn, hoặc HTTP 403	Repo chưa có trong `federation/registry.yaml` của hub, `repo_id` khác id đã đăng ký, mục đăng ký đặt pin `workflow:`, hoặc lượt chạy không nằm trên tag `kb-publish/*`	Nhờ người vận hành hub đăng ký `owner/repo` với `repo_id` của bạn và không có `workflow:`; sửa `repo_id`; publish bằng `kb publish`
Một task kết thúc `failed command-denied <command>`	Lệnh nằm ngoài allowlist	Thêm nó vào `permissions.allow` trong `.claude/settings.json` bằng tay

6. Phụ lục

6.1 Khóa config và biến môi trường

Khóa hoặc biến	Ý nghĩa
`kind`, `hub`, `repo_id`, `intake`	xem 2.2. Không bao giờ đặt token trong `hub:` hay `intake:`
`auto.enabled`, `auto.qa_interval`, `auto.idle_interval`, `auto.hotfix_base`	chế độ auto; mặc định `qa_interval` là `300` và `idle_interval` là `600` giây; `hotfix_base` rỗng = nhánh mặc định (các skill hiện giả định `main`). Bản dựng khung chỉ comment khối này; `kb auto on` ghi khối thật
`STRATA_KB_HUB`	ghi đè `hub:`; mang URL có credential trong CI (một Actions variable)
`STRATA_KB_HUB_URL`, `STRATA_KB_HTTP_TOKEN`	URL HTTP gốc và bearer token của hub: MCP và auto runner
`KB_HUB_TOKEN`, `KB_FAIL_ON_STALE`	Actions secret (chỉ hub riêng tư) và Actions variable của `kb-ticket-lint.yml` và `kb-doctor.yml`
`STRATA_KB_AUTO_TASK`	do runner đặt trên các tiến trình task; `kb auto on`, `off` và `run` từ chối khi có nó
`STRATA_KB_CLOUD`, `CLAUDE_CODE_REMOTE`	ép bật hoặc nhận diện Claude Code cloud

Mã thoát: `0` ok, `1` lỗi, `2` trích dẫn cũ (hoặc lỗi dùng lệnh); `kb auto` như ở 4.3. Các skill gọi các công cụ MCP `strata-kb`; danh sách đầy đủ nằm trong hướng dẫn hub.

6.2 `kb init` ghi gì, và gì bị git bỏ qua

Đường dẫn	Chứa	Commit
`tickets/`, `missions/`	ticket và mission plan của bạn	có
`docs/ac-quality.local.md`, `docs/review-rubric.local.md`	phần ghi đè của bạn; tạo một lần, không bao giờ làm mới	có
`.kb/*-plan/`, `.kb/index.yaml`	do `kb plan-ingest` sinh	không
`.kb/auto/`	trạng thái của auto runner (`kb init` thêm nó vào `.gitignore`)	không
`.env`	credential MCP (`kb mcp-setup` làm git bỏ qua nó; `kb init` thì không)	không

Chạy lại `kb init --kind ba` tạo các file còn thiếu, làm mới các file dựng khung khác với template và thêm các dòng ignore `.kb/*-plan/`, `.kb/index.yaml` và `.kb/auto/` vào `.gitignore`, không bao giờ xóa dòng nào. `.kb/config.yaml`, `.kb/index.yaml` và `.claude/settings.json` được bảo vệ: lần chạy thường để nguyên hai file đầu và chỉ gộp vào file thứ ba (mục của bạn được giữ). **`--force` thay thế cả ba hoàn toàn**, gồm `kind`, `repo_id` và toàn bộ `.claude/settings.json` của bạn. Không có gì dưới `tickets/` hay `missions/` bị chạm tới. Không bao giờ commit `.kb-qa/`: đó là kho Q&A của hub, được git bỏ qua trên hub.

6.3 Cổng kiểm tra những gì

Lỗi (cổng thất bại): thiếu một section bắt buộc (Summary, User Story, Background / Business context, Acceptance Criteria, Use cases, Business flow, KB context, Definition of Ready); một heading xuất hiện hai lần; hơn 10 tiêu chí chấp nhận hoặc hơn một user story (hãy tách ticket); một ref `## KB context` không phân giải được ở commit đã ghim; một trích dẫn nội dòng `[doc-id §section]` không có ref ghim đỡ lưng; change log không khớp revision. **Cảnh báo (phán đoán của bạn):** một tiêu chí chạm tới tiêu chuẩn mà không có trích dẫn; một ref đã cũ; một ref ghim mà thân ticket không bao giờ trích dẫn; thiếu dòng `> Revision:` hoặc `## Review record`; một AC quy định lệnh shell hoặc không phải Given/When/Then; thiếu hoặc rỗng một section mở rộng mà lời lẽ của story đòi hỏi.

6.4 Tri thức code trong ticket

Ngoài các tài liệu miền nghiệp vụ, hub giữ hai tài liệu cho mỗi repository sản phẩm, do workflow riêng của repository đó publish: `<repo>-code` (cấu trúc sinh tự động: `svc.*`, `db.*`, `api.*`, `int.*`, `cmd.*`, `route.*`, `page.*`, `job.*`, `sym.*`, `struct.tree`, `struct.files.*`, `struct.coverage`) và `<repo>-svc` (trách nhiệm được biên tập). Các skill BA không bao giờ tự đọc cái nào trong hai: khi một ticket cần tên service, bảng, route hay file, chúng ghi `%%TODO: verify against codebase%%` kèm một câu hỏi mở có chủ, và `/sa-ticket-ground` trả lời từ `<repo>-code`. Mọi thứ nó ghi được `kb ticket check` kiểm tra; cái tài liệu không chứng minh được nằm dưới `Open decisions` cho developer. Một tham chiếu section được viết `<repo>-code §svc.<name>`.

> **Một lưu ý.** `<repo>-svc` làm nền cho một sơ đồ. Nó không bao giờ thay thế được trích dẫn miền nghiệp vụ trong một tiêu chí chấp nhận. Một mã, định dạng, enum hay ngưỡng mã hóa một tiêu chuẩn phải đến từ một section miền nghiệp vụ đã ghim.

`struct.coverage` cho SA biết nên tin vắng mặt đến đâu, theo từng khu vực: `complete`, `partial` hoặc `not scanned` (ví dụ repo Dev thiếu `strata-kb[code]`). Vắng khỏi một scope `complete` nghĩa là chưa xây: SA ghi `<thing> in <path> [NEW: D<n>]` (file là một mục `Files:` riêng, `<path> [NEW: D<n>]`). Vắng khỏi scope `partial` hoặc `not scanned` nghĩa là chưa biết, nên nó nằm dưới `Open decisions` và cổng làm `[NEW]` ở đó thất bại. Job không bao giờ `complete` (`jobs outside framework schedulers are not detectable`). Chi tiết: Hướng dẫn cho Developer.

6.5 Quyền của chế độ auto

`kb init --kind ba` gộp quyền của nó vào `.claude/settings.json` của bạn. **Cho phép:** `git` thường ngày (add, commit, status, diff, log, fetch, pull, switch sang `auto/*`, `fix/*`, `hotfix/*`, branch, worktree, merge, `push -u origin HEAD`), `gh pr` và `gh run`, `Read`/`Edit`/`Write`, các công cụ MCP `strata-kb` và nhiều lệnh con `kb`. **Từ chối:** mọi `git push` nêu `main` hoặc `master`, force-push, refspec `+` và `:` cùng `--delete`; các dạng `git checkout`, `switch` và `branch` chuyển sang, tạo, đổi tên hoặc xóa `main` hay `master` (một id nhánh chỉ chứa từ đó, như `auto/M-domain-US1`, thì được); `git reset --hard`; `gh repo delete`, `gh run delete`; `kb init`; `rm -rf /*`; sửa `.claude/settings.json`, `.claude/settings.local.json` và `.git/**`. Các hook `Stop` và `SubagentStop` chạy `kb usage ingest-transcript --hook-stdin`; `kb usage report` biến sổ cái đó thành báo cáo chi phí (`.kb/usage/report.html`).

Báo cáo chi phí. Hook thêm khoảng 0,6 giây mỗi lượt (chi phí khởi động `kb`). `kb usage report` ghi `.kb/usage/report.html`, theo ticket, phase và model; với `--ticket <id> --md` nó in một bảng Markdown. Hook lỗi thì im lặng theo thiết kế (hook `Stop` không thể chặn một lượt kết thúc), nên nếu `.kb/usage/*.jsonl` không tăng,

hãy đọc `.kb/usage/ingest-errors.log`: đó là nơi duy nhất lỗi bên trong `kb` hiện ra. Ingest lại một transcript đã kết thúc là an toàn, vì lệnh gọi đã có trong sổ cái thì bị bỏ qua: `kb usage ingest-transcript <path>` nạp bù một transcript. Chỉ chạy nó trên phiên đã kết thúc: một lệnh gọi còn đang stream sẽ bị lưu với số token đầu ra chưa đầy đủ, và một dòng đã lưu không bao giờ thay đổi. `report.html` được tạo lại từ sổ cái ở mỗi lần chạy, nên sổ cái mới là bản ghi, không phải báo cáo. **Phân tầng model** chỉ là khuyến nghị, không có gì trong `kb` bắt buộc. Giá model lấy từ `usage-prices.yaml` (đi kèm gói, ghi đè theo từng model ở `.kb/usage-prices.yaml`), có `effective_date`. Model không có dòng giá được báo là `unpriced`, không bao giờ đoán, và các lệnh gọi fast-mode được tính theo giá chuẩn và đánh dấu, nên tổng có chứa chúng chỉ là cận dưới. Đổi model ở ranh giới phase, không bao giờ giữa một phase: đổi model làm mất prompt cache và phải trả một lần ghi cache mới, nên vài lượt model rẻ giữa một phiên model mạnh có thể tốn **nhiều hơn** không phân tầng. Kiểm tra kết quả bằng `kb usage report`, vốn chia chi tiêu theo model.

6.6 Quy tắc Q&A của hub liên quan tới bạn

Câu hỏi và câu trả lời nằm trong kho Q&A của hub, chỉ được ghi qua `kb_question_ask` (Dev) và `kb_question_answer` (BA). `asked_by` và `answered_by` là tên do người gọi tự gõ (các skill truyền `git config user.name`): để theo dõi, không phải xác thực. Giới hạn: 4000 ký tự cho một câu hỏi hoặc câu trả lời, 1000 cho bối cảnh, 100 cho tên, tối đa 50 section thay đổi cho mỗi câu trả lời và 50 câu hỏi mở cho mỗi ticket.